Projecte ECS + ECR

Noms: Cristian Martínez, Lluc Flores & Roc Siñol
Professor: Pedro Durán Diez i Carlos Gutierrez
[Repositori Github]

ÌNDEX

1. Introducció i Objectius.....2
2. Arquitectura de la solució.....3
3. Implementació del projecte.....3
3.1 Fase 1: Prova de Concepte (Desplegament Manual i Verificació).....4
3.2 Fase 2: Automatització amb Terraform (IaC).....7
3.3 Desplegament de l'Aplicació i Resultat Final.....9
4. Dificultats.....10
5. Treball futur.....11

1. Introducció i Objectius

L'objectiu principal en aquest projecte és el desplegament d'un videojoc, o una web, en una infraestructura al núvol públic (AWS) amb alta disponibilitat i escalable. Per aconseguir això, hem fet servir tant la GUI d'AWS com Terraform adoptant la metodologia IaC (Infraestructure as Code).

L'aplicació del videojoc s'executa a través de contenidors Docker, controlats per Amazon ECS (Elastic Container Service), amb instàncies EC2. D'aquesta manera ens assegurem que el servei pugui recuperar-se automàticament davant d'errors.

2. Arquitectura de la solució

Per aconseguir això, hem dissenyat una arquitectura que utilitza els següents components principals:

- **Terraform:** Com a eina principal per definir tota la infraestructura (xarxa, computació i seguretat) mitjançant codi (IaC).

- **Docker i AWS ECR:** Contenidorització de l'aplicació i emmagatzematge de la imatge Docker (tag v1.0) en un registre privat (Elastic Container Registry).

- **AWS ECS (Cluster)**: L'orquestrador que gestiona l'execució i el cicle de vida dels contenidors.

- **Auto Scaling Group (ASG)**: Configurat amb instàncies EC2 (AMI optimitzada per ECS) per assegurar que el servei pugui escalar o recuperar-se automàticament davant d'errors.

- **Security Groups**: Configuració de firewall per permetre tràfic HTTP (port 80) i rang dinàmic per l'agent d'ECS.

3. Implementació del projecte

El desenvolupament s'ha dividit en dues grans fases per assegurar el correcte funcionament abans de l'automatització total.

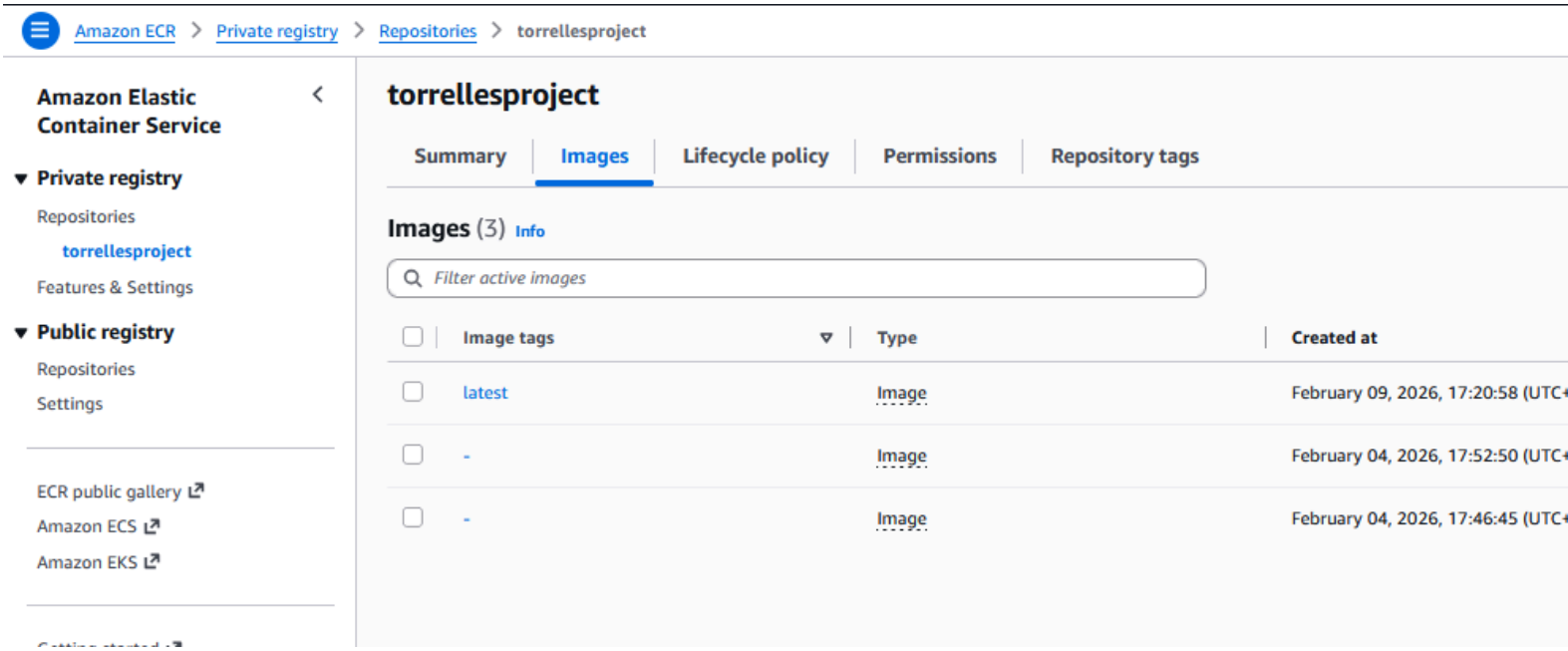
3.1 Fase 1: Prova de Concepte (Desplegament Manual i Verificació)

Inicialment, vam realitzar el desplegament manual a la consola d'AWS per validar l'arquitectura:

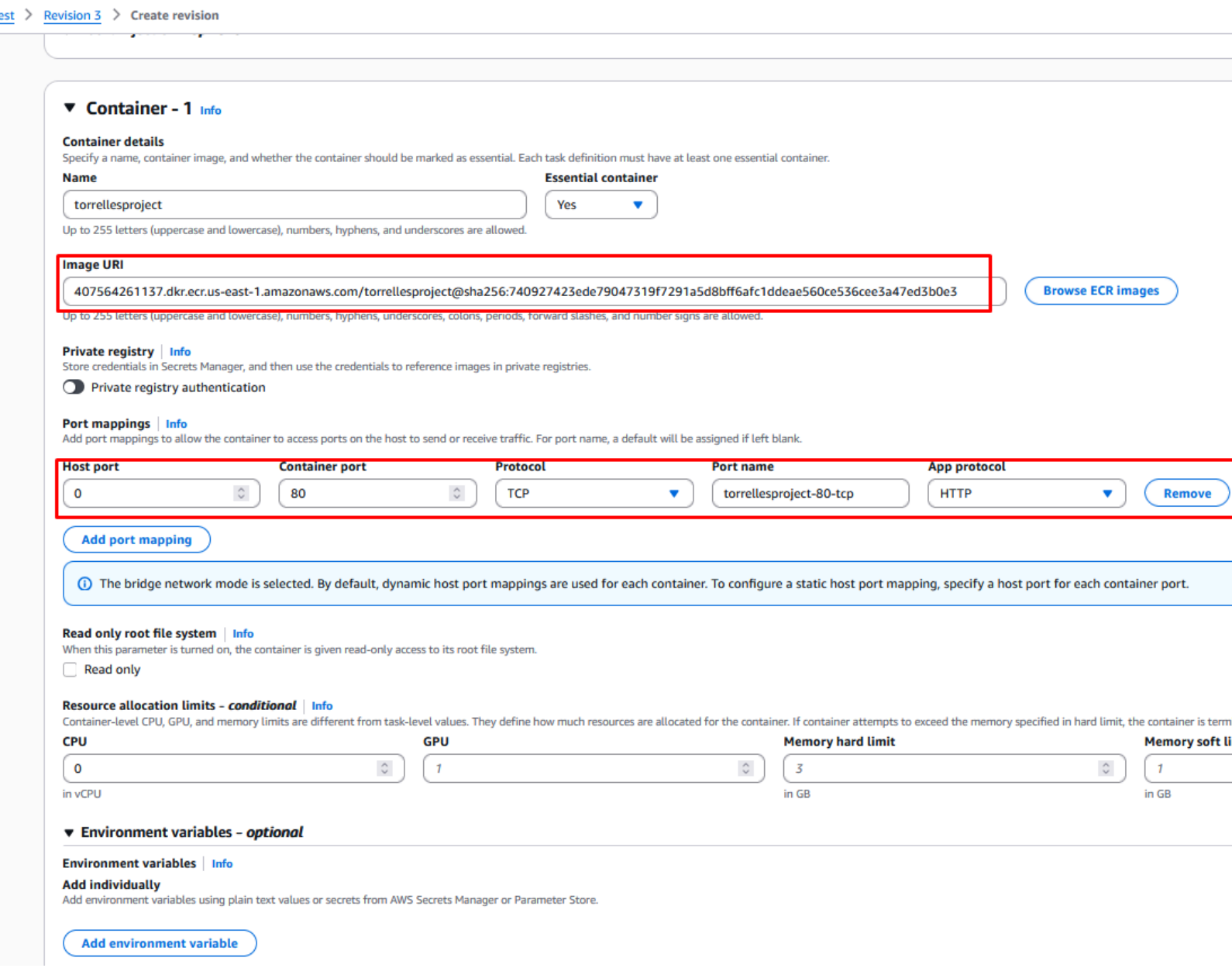
- **Verificació local del contenidor:** Comprovació inicial del funcionament de la imatge Docker a la instància EC2 manual abans de pujar-la al núvol.

```
[ec2-user@ip-172-31-78-182 ~]$ sudo docker run -d -p 80:80 torrellesproject
2fbf366bc7a8c0b809de3f4a825d0a764758db3ddb5f1c4fe0e5d0238e65a902
[ec2-user@ip-172-31-78-182 ~]$ docker ps
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar%2Frun%2Fdocker.sock/v1.44/con
[ec2-user@ip-172-31-78-182 ~]$ sudo docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS          PORTS                                   NAMES
2fbf366bc7a8   torrellesproject   "/docker-entrypoint.…"   10 seconds ago   Up 9 seconds    0.0.0.0:80->80/tcp, :::80->80/tcp   objective_gould
```

- **Repositori ECR:** Imatge Docker empaquetada i pujada al registre privat d'Amazon ECR, llesta per ser desplegada.



- **Definició de la Tasca (Task Definition):** Configuració del contenidor amb mode de xarxa "Bridge" i **mapeig dinàmic del port (Host: 0, Contenidor: 80)** per evitar conflictes quan l'Auto Scaling Group aixequi múltiples instàncies. *task_definition_ports)*



- **Infraestructura d'Auto Scaling:** Instàncies EC2 aprovisionades automàticament pel Capacity Provider del clúster per allotjar les càrregues de treball.

Instances (6) Info

Find Instance by attribute or tag (case-sensitive) | All states

Instance state = running | Clear filters

Name	Instance ID	Instance state	Instance type	Status chec
ECS Instance - cluster-netflix	i-0c6272a9ff8dfbb32	Running	t3.small	3/3 chec
ECS Instance - cluster-netflix	i-038aa7b45745530c9	Running	t3.small	3/3 chec
ECS Instance - cluster-netflix	i-056afb479c7f5b597	Running	t3.small	3/3 chec
Linux01_Docker_Proyecto	i-0db708d61c3ea7aad	Running	t3.micro	3/3 chec
ECS Instance - cluster-netflix	i-05b4ae10084477664	Running	t3.small	3/3 chec
ECS Instance - cluster-netflix	i-080679e40415698f1	Running	t3.small	3/3 chec

Select an instance

- **Configuració de Seguretat:** Habilitació del trànsit HTTP entrant (port 80) al Security Group des de l'origen 0.0.0.0/0.

[Alt+S] Ask Amazon Q

nstancias-sg

sg-09d4fed1a036864e2 - ecs-instancias-sg

Details

Security group name	Security group ID	Description
ecs-instancias-sg	sg-09d4fed1a036864e2	Solo acceso desde el ALB
Owner	**Inbound rules count**	**Outbound rules count**
407564261137	3 Permission entries	1 Permission entry

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - *new* | Tags

Inbound rules (3)

Search

Name	Security group rule ID	IP version	Type	Protocol	Port ran
-	sgr-0e9c3041ef3bff79f	IPv4	SSH	TCP	22
-	sgr-044b15b338df2f70a	-	Custom TCP	TCP	0
-	sgr-0ad66c488cc8cc5b5	IPv4	HTTP	TCP	80

3.2 Fase 2: Automatització amb Terraform (IaC)

Un cop validat el model, vam aplicar la filosofia DevOps automatitzant la infraestructura. El codi s'ha modularitzat en diferents fitxers (provider, network, compute, app). Un exemple de la configuració de xarxa lliure d'errors tipogràfics:

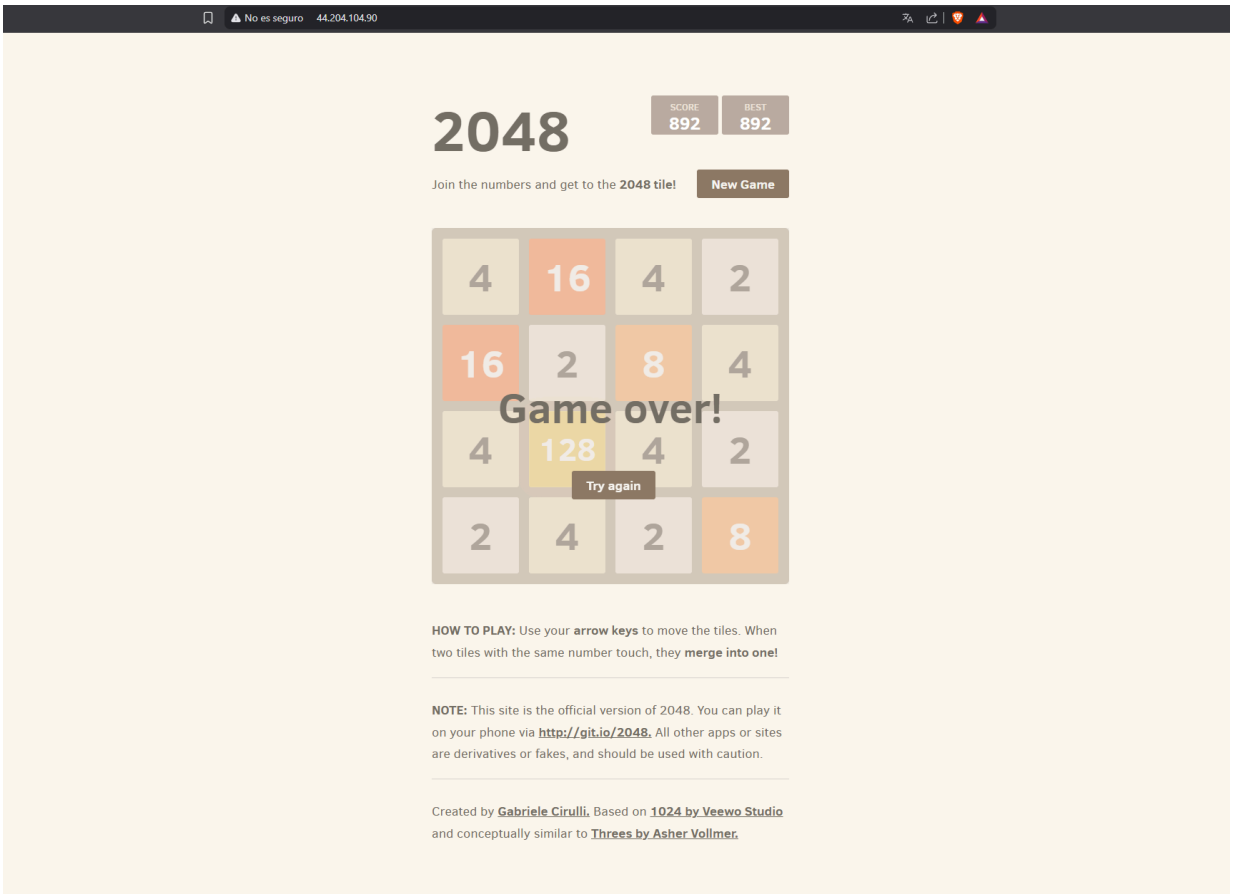
network.tf

```
resource "aws_security_group" "ecs_sg" {
  name        = "${var.project_name}-sg"
  description = "Security Group per a instancies ECS"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    description = "Acces Web App"
    from_port   = var.app_port
    to_port     = var.app_port
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Rang ports dinamics ECS"
    from_port   = 32768
    to_port     = 65535
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
lfl6267@A27-23:~/optcloud/proyecto-ecs-2048_Torrelles_Team$ terraform apply --auto-approve
        + {
            + cidr_blocks      = [
                + "0.0.0.0/0",
              ]
            + description      = "SSH para mantenimiento (Opcional)"
            + from_port        = 22
            + ipv6_cidr_blocks = []
            + prefix_list_ids  = []
            + protocol         = "tcp"
            + security_groups  = []
            + self             = false
            + to_port          = 22
          },
      ]
    + name                   = "asix2-ecs-2048-sg"
    + name_prefix            = (known after apply)
    + owner_id               = (known after apply)
    + revoke_rules_on_delete = false
    + tags                   = {
        + "Name" = "asix2-ecs-2048-sg"
      }
    + tags_all               = {
        + "Name" = "asix2-ecs-2048-sg"
      }
    + vpc_id                 = "vpc-01309cc25a1591f25"
  }

Plan: 7 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ecr_url = (known after apply)
aws_ecr_repository.app_repo: Creating...
aws_ecs_cluster.main: Creating...
aws_security_group.ecs_sg: Creating...
aws_ecr_repository.app_repo: Creation complete after 1s [id=asix2-game-repo]
aws_ecs_task_definition.app_task: Creating...
aws_ecs_task_definition.app_task: Creation complete after 1s [id=asix2-ecs-2048-task]
aws_security_group.ecs_sg: Creation complete after 5s [id=sg-044fc2383e9176f06]
aws_ecs_cluster.main: Still creating... [00m10s elapsed]
aws_ecs_cluster.main: Creation complete after 12s [id=arn:aws:ecs:us-east-1:848869408721:cluster/asix2-ecs-2048-cluster]
aws_ecs_service.app_service: Creating...
aws_launch_template.ecs_lt: Creating...
aws_ecs_service.app_service: Creation complete after 1s [id=arn:aws:ecs:us-east-1:848869408721:service/asix2-ecs-2048-cluster/asix2-ecs-2048-svc]
aws_launch_template.ecs_lt: Creation complete after 6s [id=lt-02d5a336accc858ce]
aws_autoscaling_group.ecs_asg: Creating...
aws_autoscaling_group.ecs_asg: Still creating... [00m10s elapsed]
aws_autoscaling_group.ecs_asg: Creation complete after 17s [id=asix2-ecs-2048-asg]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

Outputs:

ecr_url = "848869408721.dkr.ecr.us-east-1.amazonaws.com/asix2-game-repo"
lfl6267@A27-23:~/optcloud/proyecto-ecs-2048_Torrelles_Team$
```

3.3 Desplegament de l'Aplicació i Resultat Final

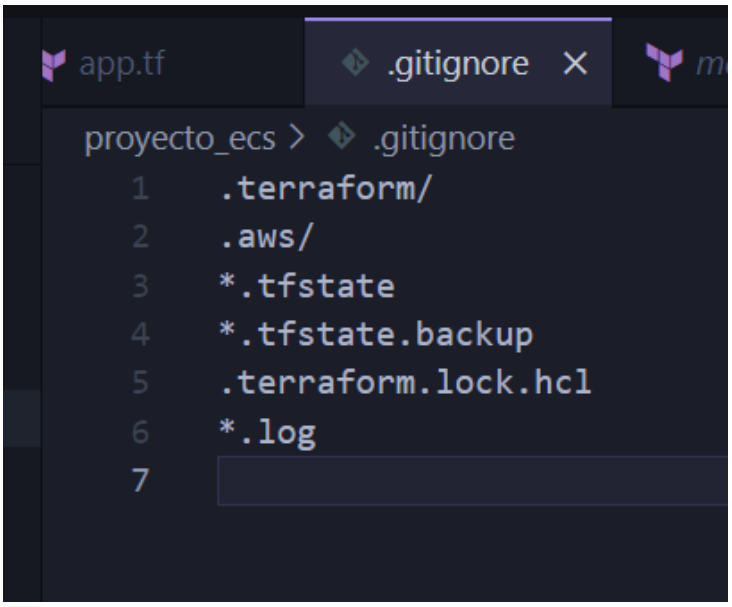
Un cop aixecada la infraestructura, el flux definitiu ha estat fer login a l'ECR, construcció de la imatge i etiquetatge amb la versió exigida (v1.0).



4. Dificultats

Durant el desenvolupament del projecte, hem trobat i solucionat diversos reptes tècnics:

- **Límit de grandària a GitHub:** Al fer el primer push, Git fallava perquè la carpeta oculta .terraform superava els 100MB permesos. Ho vam solucionar creant un fitxer .gitignore adequat i netejant la memòria cau de Git (git rm -rf --cached .).



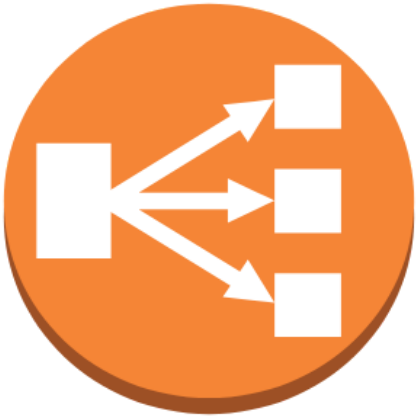
- **Caducitat de Credencials (AWS Learner Lab):** Al estar en un entorn acadèmic, el Session Token d'AWS caducava diàriament. Vam establir un protocol d'equip per actualitzar l'arxiu ~/.aws/credentials a l'inici de cada sessió.

- **Conflicte de Ports a l'Auto Scaling:** Inicialment vam mapejar el port 80 del Host al 80 del contenidor. Quan l'ASG intentava aixecar més d'una tasca a la mateixa instància EC2, el port ja estava ocupat. Ho vam investigar i solucionar aplicant el Dynamic Port Mapping (Host port: 0) a la Task Definition.

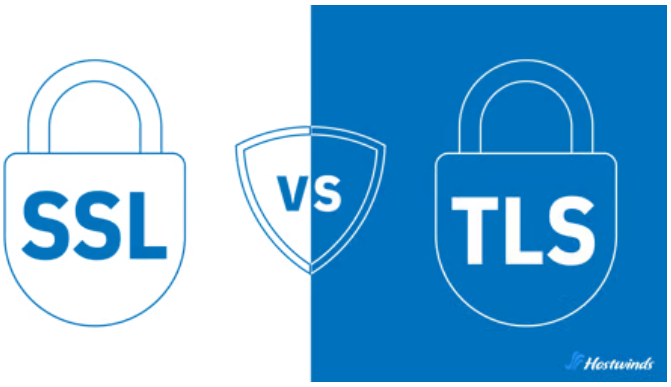
5. Treball futur

Per millorar l'arquitectura en un entorn de producció real, proposem les següents millores:

- **Implementació d'un Application Load Balancer (ALB):** Actualment accedim directament per la IP de la instància EC2. Un ALB permetria repartir la càrrega entre totes les instàncies de l'ASG automàticament.



- **Certificats TLS/SSL:** Integració amb AWS Certificate Manager (ACM) per xifrar el trànsit web mitjançant HTTPS.



- **Pipeline CI/CD:** Implementar GitHub Actions perquè cada cop que fem un push a la branca main, Terraform apliqui els canvis automàticament i Docker construeixi una nova imatge cap a l'ECR.